Intelligent Audio Enhancement in Noisy Environments Using Analog Electronics, Digital Signal Processing, and Machine Learning

A Project Based Learning Report Submitted in partial fulfilment of the requirements for the award of the degree

of

Bachelor of Technology

in The Department of Electronic and Communications

Signal Processing, Analog and Electronic Circuit Design, Machine Learning and 25EC2104E, 25EC1302E, 25SC2107E

Submitted by

2510040049: B. Muthyala Raju

2510040082: K. Vishnu Vardhan

2510040041: K. Vijender Reddy

2510040073: P. Phanindra Reddy

Under the guidance of

Dr. Ashwini Varma, Vivek Pathak, Dr. A. Bhuvaneshwari, Dr. Sudharsana Rao



Department of Electronics and Communication Engineering

Koneru Lakshmaiah Education Foundation, Aziz Nagar

Aziz Nagar – 500075

OCT - 2026.

Abstract

Environmental noise is a major challenge in applications such as classrooms, offices, and public spaces, where unwanted sounds can interfere with speech clarity and listening quality. Common sources of noise include student conversations, fans, air-conditioning systems, and traffic. Conventional filtering techniques may reduce certain noise components; however, when speech and noise occupy overlapping frequency ranges, fixed filtering alone may also attenuate useful speech information. This project proposes an integrated audio analysis and enhancement system that combines Analog Electronics, Digital Signal Processing (DSP), and Machine Learning (ML) to demonstrate a structured approach to processing audio signals in noisy environments.

The proposed system, named Smart Audio Lab, uses a Python-based interactive dashboard to visualize and analyze audio signals. In the current software prototype, the system supports synthetic audio simulation and on-demand microphone input, subject to local device permissions. The acquired audio samples are processed using DSP techniques, including DC offset removal, Butterworth low-pass filtering, and a simple energy gate. Waveform and Fast Fourier Transform (FFT) visualizations are used to examine the time-domain and frequency-domain characteristics of the input and processed signals. Additional audio features, such as root mean square (RMS) energy, zero-crossing rate, spectral centroid, and spectral roll-off, are extracted to support environmental sound analysis.

Machine Learning is incorporated through a demonstration Random Forest classifier that predicts possible environmental categories using synthetic feature examples. This component illustrates how extracted audio features can support environmental sound classification and how environmental context could potentially guide future audio-processing strategies. The analog hardware pathway is proposed using an electret microphone, microphone biasing circuit, LM358 operational amplifier, analog low-pass filter, and ESP32 microcontroller with an analog-to-digital converter (ADC). This arrangement is intended to provide a practical interface between real-world sound acquisition and the digital processing system.

The primary research focus is the integration of analog signal conditioning, DSP-based analysis, and ML-based environmental classification within a single, explainable, and low-cost framework. The dashboard distinguishes simulated signals from microphone input and proposed hardware operation, enabling software demonstration before the physical circuit is completed. Future work will focus on hardware integration, validation using labelled environmental audio datasets, quantitative evaluation of classification performance, and measurement of audio enhancement effectiveness under different noise conditions. The proposed system provides an educational and experimental platform for understanding interdisciplinary audio processing and exploring intelligent audio enhancement in noisy environments.

Keywords: Audio Enhancement, Analog Electronics, Digital Signal Processing, Machine Learning, Environmental Sound Classification, FFT, Random Forest, ESP32, Streamlit Dashboard.

List of Figures

Figure No. Figure Title

Figure 1.1 Overall System Architecture of the Proposed Smart Audio Enhancement System

Figure 1.2 Smart Classroom Environmental Noise Scenario

Figure 1.3 Proposed Microphone Signal-Conditioning Circuit

Figure 1.4 LM358-Based Microphone Preamplifier Circuit

Figure 1.5 Anti-Aliasing Filter Circuit

Figure 1.6 ESP32-Based Analog-to-Digital Conversion (ADC) Architecture

Figure 1.7 Audio Signal Processing Workflow

Figure 1.8 Input Audio Signal Waveform

Figure 1.9 Frequency Spectrum of the Input Audio Signal Using FFT

Figure 1.10 Comparison of Input and Processed Audio Waveforms

Figure 1.11 Comparison of Input and Processed Audio Frequency Spectra

Figure 1.12 Audio Feature Extraction Workflow

Figure 1.13 Random Forest-Based Environmental Sound Classification Workflow

Figure 1.14 Main Interface of the Smart Audio Lab Python Dashboard

Figure 1.15 Machine Learning Prediction Dashboard

Figure 1.16 Integration of Analog Electronics, Digital Signal Processing, and Machine Learning

Figure 1.17 Original and Processed Audio Comparison Interface

Figure 1.18 Proposed Hardware Prototype for Smart Audio Enhancement

Figure 1.19 Complete Software and Hardware Integration Workflow

Figure 1.20 Future Evaluation and Validation Plan

List of Tables

Table No Table Name

Table 1.1 — Common Environmental Noise Sources and Their Effects on Speech

Table 1.2 — Objectives and Expected Outcomes of the Project

Table 2.1 — Functional Roles of Analog Electronics, Digital Signal Processing, and Machine Learning

Table 2.2 — Hardware Components Required for the Proposed Prototype

Table 2.3 — Software Tools and Libraries Used in the Smart Audio Lab Dashboard

Table 3.1 — Digital Signal Processing Techniques and Their Functions

Table 3.2 — Audio Features Extracted for Environmental Sound Analysis

Table 4.1 — Environmental Sound Categories Considered in the Demonstration

Table 5.1 — Comparison of Synthetic Simulation, Laptop Microphone Capture, and Proposed Hardware Input

Table 6.1 — Summary of Selected Literature on Environmental Sound Classification

Table 6.2 — Research Gaps and the Proposed Project's Focus

Table 7.2 — Proposed Evaluation Metrics for Future System Validation

Table of Contents

1. INTRODUCTION

1.1 Background of the Project

1.2 Problem Statement

1.3 Objectives of the Project

1.4 Scope of the Project

2. METHODOLOGY

2.1 Proposed System Architecture

2.2 Analog Electronics and Signal Conditioning

2.3 Digital Signal Processing Techniques

2.4 Audio Feature Extraction

2.5 Machine Learning Using Random Forest

2.6 Smart Audio Lab Software Implementation

2.7 Hardware and Software Requirements

3. EXPERIMENTS

3.1 Experimental Setup

3.2 Audio Signal Generation and Capture

3.3 Waveform and Frequency Spectrum Analysis

3.4 Digital Audio Processing

3.5 Audio Feature Extraction and ML Prediction

3.6 Original and Processed Audio Comparison

4. RESULTS

4.1 Audio Signal Analysis Results

4.2 Digital Signal Processing Observations

4.3 Machine Learning Demonstration Results

4.4 Discussion and Limitations

5. CONCLUSION AND FUTURE WORK

5.1 Conclusion

5.2 Current Limitations

5.3 Future Enhancements

Intelligent Audio Enhancement in Noisy Environments Using Analog Electronics, Digital Signal Processing, and Machine Learning

1. INTRODUCTION

1.1 Background of the Project

Audio communication plays an important role in education, workplaces, public spaces, and everyday life. However, the quality and clarity of audio signals can be affected by environmental noise such as conversations, traffic, fans, air-conditioning systems, and other unwanted sounds. These disturbances can make speech difficult to understand, particularly in environments where clear communication is essential, such as classrooms, meeting rooms, and laboratories.

Traditional audio systems often use microphones and amplifiers to capture and strengthen sound signals. Although amplification increases the signal level, it does not necessarily distinguish useful speech from unwanted background noise. In some cases, amplification may increase both the desired speech and the background noise. Therefore, an effective audio enhancement system should combine appropriate signal acquisition, signal conditioning, digital processing, and intelligent analysis.

The project **"Intelligent Audio Enhancement in Noisy Environments Using Analog Electronics, Digital Signal Processing, and Machine Learning"** proposes an integrated approach to demonstrating these functions. The project is implemented through a Python-based application called **Smart Audio Lab**, which provides a graphical interface for audio simulation, microphone capture, digital signal processing, audio feature extraction, and machine-learning demonstration.

The proposed approach combines three major technical areas. First, analog electronics can be used to condition a microphone signal before digitization. This may involve amplification, biasing, and filtering, depending on the selected microphone and hardware configuration. Second, digital signal processing (DSP) techniques can be applied to analyze and modify the digitized audio signal. Third, machine-learning methods can use extracted audio features to demonstrate environmental sound classification.

The current Smart Audio Lab application demonstrates audio processing using synthetic signals and short, on-demand recordings from a laptop microphone. It provides waveform and frequency-spectrum visualizations, applies basic digital processing techniques, and extracts selected audio features. A Random Forest model trained on synthetic feature examples is included to demonstrate the machine-learning workflow.

The physical analog circuit and ESP32-based acquisition pathway form part of the proposed hardware direction. Their inclusion in the system design does not, by itself, establish that a physical prototype has been assembled or experimentally validated. The software and hardware components must therefore be described according to their actual implementation status.

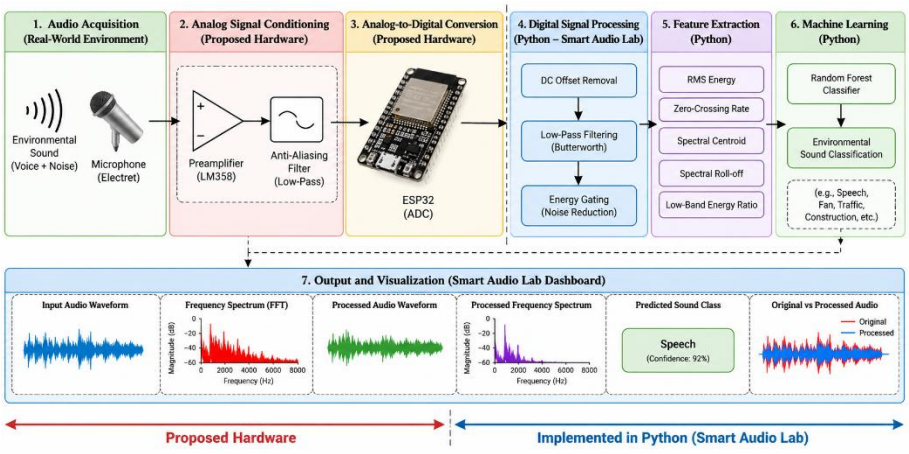


Figure 1.1 here: Overall System Architecture of the Proposed Smart Audio Enhancement System

1.2 Problem Statement

Environmental noise can reduce speech intelligibility and affect the reliability of audio-based communication systems. A microphone may capture both the desired speech and surrounding noise, while a conventional amplifier may increase the levels of both. Consequently, amplification alone is not sufficient to provide effective noise reduction.

Digital signal processing offers methods for modifying audio signals, but simple filters and energy-based methods have limitations. For example, a low-pass filter can reduce high-frequency components, but it may also remove useful speech information. Similarly, an energy gate can suppress low-energy sections of an audio signal, but quiet speech may also be attenuated. The effectiveness of these techniques depends on the characteristics of the input audio and the selected processing parameters.

Machine learning provides another approach by analyzing features extracted from audio signals and identifying patterns associated with different sound categories. However, the usefulness of a classifier depends on the quality and representativeness of its training data and on suitable evaluation. A model trained only on synthetic feature examples cannot be assumed to perform accurately on real environmental recordings without further testing.

A further challenge is the integration of analog electronics, DSP, and machine learning into a single demonstrator. These areas are often studied separately, making it difficult to observe how microphone signal conditioning, digital processing, audio analysis, and classification relate to one another.

Therefore, the main problem addressed by this project is the development of an explainable, low-cost demonstration platform that brings together audio signal analysis, basic digital enhancement techniques, feature extraction, and machine-learning concepts while providing a pathway toward future hardware integration and systematic evaluation.

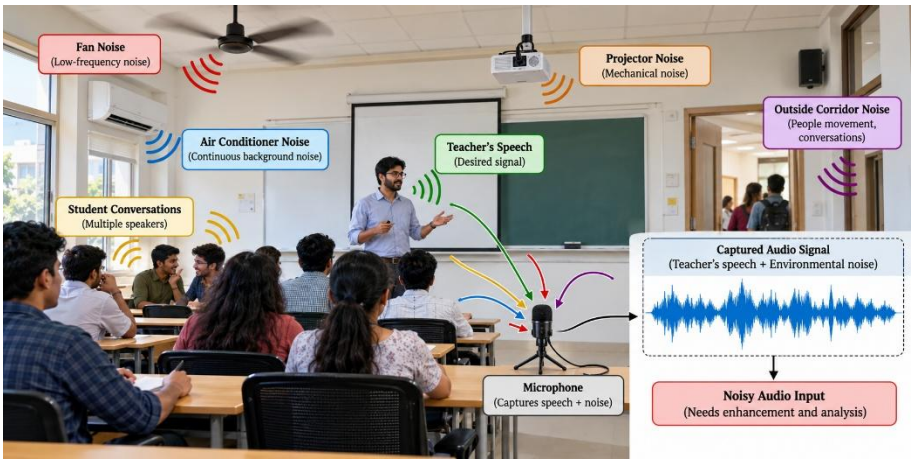


Figure 1.2 here: Smart Classroom Environmental Noise Scenario

1.3 Objectives of the Project

The primary objective of this project is to develop **Smart Audio Lab**, a Python-based demonstration platform for studying audio enhancement in noisy environments through analog electronics, digital signal processing, and machine learning.

The specific objectives are:

1. To study the role of microphone signal conditioning and analog electronics in an audio acquisition system.
2. To design a proposed hardware architecture that can accommodate a microphone, signal-conditioning circuit, and ESP32-based analog-to-digital conversion pathway.
3. To implement basic digital audio processing techniques, including DC offset removal, Butterworth low-pass filtering, and a simple energy gate.
4. To visualize audio signals in the time domain and analyze their frequency content using the Fast Fourier Transform (FFT).
5. To extract audio features such as root mean square (RMS) energy, zero-crossing rate, spectral centroid, spectral roll-off, and low-band energy ratio.
6. To demonstrate the use of a Random Forest classifier with synthetic audio-feature examples.
7. To provide a graphical dashboard for examining input audio, processed audio, extracted features, and demonstration predictions.
8. To establish a foundation for future evaluation using recorded environmental sounds and a physically implemented hardware prototype.

The current project emphasizes demonstration and integration rather than claiming a fully validated noise-cancellation system. Performance evaluation using real environmental datasets, physical hardware measurements, and quantitative comparisons remains necessary for establishing practical effectiveness.

Table 1.1: Objectives and Expected Outcomes of the Project

Objective Number	Project Objective	Expected Outcome	Implementation Status
1	To study the role of microphone signal conditioning and analog electronics.	Understanding of microphone biasing, pre-amplification, and filtering concepts.	**Completed** (Study and design)
2	To design a proposed hardware architecture with microphone, signal-conditioning circuit, and ESP32-based ADC pathway.	Block diagram and circuit design for the proposed hardware system.	**Proposed** (Design only, not physically implemented)
3	To implement digital audio processing techniques (DC offset removal, low-pass filtering, and energy gating).	Processed audio signal with reduced background noise.	**Completed** (Implemented in Python)
4	To visualize audio signals in time and frequency domains using waveform and FFT analysis.	Waveform and frequency spectrum plots for input and processed audio.	**Completed** (Implemented in Python)
5	To extract audio features from the processed signals (RMS, zero-crossing rate, spectral centroid, spectral roll-off, low-band energy ratio).	Numerical feature values and feature visualization for different audio inputs.	**Completed** (Implemented in Python)
6	To demonstrate Random Forest-based environmental sound classification.	Predicted environmental sound class with confidence score.	**Completed** (Implemented in Python with synthetic examples)
7	To develop a graphical dashboard (Smart Audio Lab) for audio analysis, processing, and ML demonstration.	User-friendly interface for input selection, visualization, processing, and prediction.	**Completed** (Implemented using Streamlit)
8	To establish a foundation for future evaluation and real-world validation.	Planned methodology for testing with real environmental recordings and hardware prototype.	**Future Work** (To be implemented and evaluated)

1.4 Scope of the Project

The scope of the project covers the software demonstration of audio analysis and processing, the study of relevant analog signal-conditioning concepts, and the design of a possible hardware integration pathway.

1.4.1 Audio Input and Signal Visualization

The Smart Audio Lab application supports synthetic audio simulation and short, on-demand laptop microphone capture. The acquired or generated audio signal can be displayed as a waveform to examine how its amplitude changes over time. Frequency-spectrum visualization using the FFT allows the user to observe the distribution of frequency components in the signal.

The source of each audio input should be identified as synthetic simulation, laptop microphone capture, or physical hardware input, as applicable. This distinction helps users understand the origin of the signal being analyzed.

1.4.2 Digital Signal Processing

The software demonstrates three basic signal-processing operations:

- **DC offset removal:** Reduces a constant offset in the sampled signal.
- **Butterworth low-pass filtering:** Attenuates frequency components above the selected cutoff region.
- **Energy gating:** Reduces signal sections whose energy falls below a selected threshold.

The processed signal can be compared with the original signal using waveform and frequency-spectrum plots. These comparisons help illustrate the effects of the selected operations. However, these basic methods do not guarantee the removal of all environmental noise, and they may also attenuate useful speech components.

1.4.3 Audio Feature Extraction and Machine Learning

The application extracts several features that describe different characteristics of an audio signal. RMS energy provides a measure of signal magnitude, while the zero-crossing rate indicates how frequently the signal changes sign. Spectral centroid describes the approximate centre of the signal's spectral distribution, spectral roll-off identifies a frequency below which a selected proportion of spectral energy is concentrated, and low-band energy ratio measures the relative energy in a specified lower-frequency region.

These features are used to demonstrate the connection between audio analysis and machine learning. A Random Forest classifier trained on synthetic feature examples provides an illustrative prediction workflow. Its predictions should not be interpreted as proof of reliable real-world environmental sound classification because the model has not been established as a validated classifier for real-world recordings.

1.4.4 Proposed Hardware Integration

The proposed hardware pathway includes a microphone, an analog signal-conditioning stage, an analog-to-digital conversion stage, and an ESP32-based interface. Depending on the final design, analog conditioning may include microphone biasing, preamplification, and anti-aliasing filtering.

The ESP32 pathway is intended to provide sampled audio data to the software through a serial interface. The planned data format consists of newline-terminated CSV blocks containing integer ADC samples, subject to compatible firmware and hardware implementation.

The hardware diagrams in this report should be identified as **proposed circuits or architectures** unless the corresponding circuits have been physically assembled and verified. Similarly, the report should not claim measured hardware performance unless measurements have actually been taken.

1.4.5 Limitations and Future Scope

The current demonstration has several limitations. The Random Forest model uses synthetic feature examples rather than a validated dataset of real environmental recordings. The basic DSP techniques may attenuate speech along with noise. The application supports on-demand microphone capture and block-based hardware input as designed; it should not be described as a fully real-time hardware system unless continuous operation and its timing performance have been demonstrated.

Future work may include constructing and testing the analog front end, integrating the ESP32 acquisition pathway, collecting representative environmental audio, training and evaluating the classifier on suitable real recordings, and comparing processed audio using objective metrics such as signal-to-noise ratio (SNR) where valid reference signals are available. Speech intelligibility and unwanted speech attenuation should also be considered when evaluating the effectiveness of enhancement techniques.

In summary, the project provides a unified educational demonstrator for exploring the relationship between analog electronics, digital signal processing, and machine learning in audio applications. It establishes a software foundation and a proposed hardware pathway for further implementation and evaluation without assuming performance results that have not yet been measured.

2. METHODOLOGY

2.1 Proposed System Architecture

The methodology of the project, *Intelligent Audio Enhancement in Noisy Environments Using Analog Electronics, Digital Signal Processing, and Machine Learning*, is based on integrating three major areas: analog electronics, digital signal processing (DSP), and machine learning (ML). The system is designed to demonstrate how an audio signal can be acquired, analyzed, processed, and classified using a combination of hardware concepts and software tools.

The proposed system consists of an audio acquisition stage, an analog signal-conditioning stage, an analog-to-digital conversion (ADC) stage, a digital signal-processing stage, an audio feature-extraction stage, and a machine-learning stage. The final output is presented through a Python-based graphical dashboard named **Smart Audio Lab**.

The current software implementation supports synthetic audio generation and short, on-demand recording through a laptop microphone. The captured or generated audio can be displayed, processed, and analyzed through the dashboard. A Random Forest classifier demonstrates the classification workflow using synthetic feature examples.

The hardware pathway is designed around a microphone, a proposed analog signal-conditioning circuit, and an ESP32-based ADC interface. This pathway represents the intended hardware integration approach and should be treated as proposed unless the circuit and data-acquisition process have been physically implemented and tested.

The overall methodology follows these stages:

1. Audio signal generation or acquisition.
2. Analog signal conditioning in the proposed hardware pathway.
3. Analog-to-digital conversion in the proposed hardware pathway.
4. Digital signal processing using Python.
5. Audio feature extraction.
6. Demonstration of machine-learning-based sound classification.
7. Visualization and comparison through the Smart Audio Lab dashboard.

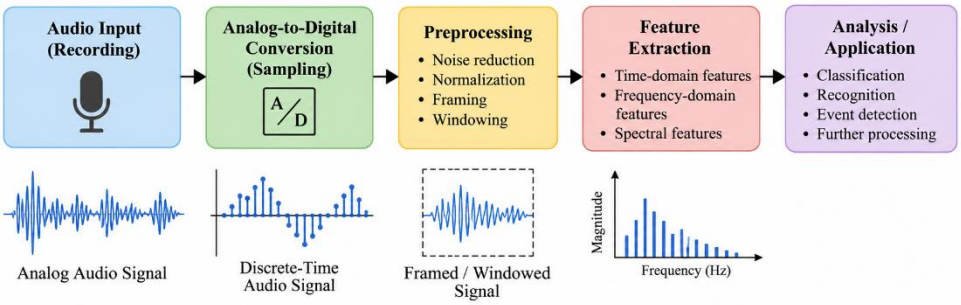


Figure 2.1: Audio Signal Processing Workflow.

2.2 Analog Electronics and Signal Conditioning

Analog electronics form the front end of the proposed hardware system. Before an analog audio signal is digitized, it may require suitable biasing, amplification, filtering, and voltage-level conditioning. These operations help prepare the signal for the ADC and can influence the quality of the acquired samples.

A microphone converts acoustic pressure variations into an electrical signal. The characteristics of the output depend on the microphone type. For example, an electret microphone capsule generally requires appropriate biasing and may require a preamplifier to produce a usable signal level.

The proposed analog front end consists of the following functional blocks:

- Microphone input.
- Microphone biasing, where required.
- Preamplifier stage.
- Low-pass anti-aliasing filter.
- ADC input interface.

The exact circuit configuration depends on the selected microphone, supply voltage, ADC characteristics, and expected input-signal range.

2.2.1 Microphone Signal Acquisition

The microphone is the initial sensing element of the proposed hardware system. It converts sound waves into an electrical signal containing the desired speech and surrounding environmental sounds.

The electrical output can contain useful speech, background noise, and unwanted interference. The microphone's sensitivity, frequency response, placement, and surrounding environment influence the captured signal.

In the current software implementation, short audio recordings can be obtained directly through a laptop microphone. This allows the dashboard's audio-processing functions to be demonstrated without requiring the proposed external analog circuit.

2.2.2 LM358-Based Microphone Preamplifier

A preamplifier increases the amplitude of a weak microphone signal before subsequent processing. An LM358 operational amplifier is one possible component for a proposed low-cost signal-conditioning circuit.

The amplifier must be designed according to the microphone's output level, supply voltage, desired gain, and output range. Suitable biasing and coupling components may also be required, depending on the microphone and circuit topology.

The gain of a non-inverting operational-amplifier configuration can be expressed as:

$$A_v = 1 + \frac{R_f}{R_g}$$

where:

- A_v is the voltage gain.
- R_f is the feedback resistance.
- R_g is the resistance connected between the inverting input and the reference node.

This equation applies to the specified non-inverting configuration; the actual gain depends on the circuit design.

Amplification increases the signal level but does not inherently separate speech from noise. If both speech and noise are present at the input, both may be amplified. Therefore, amplification should be followed by appropriate signal processing where necessary.

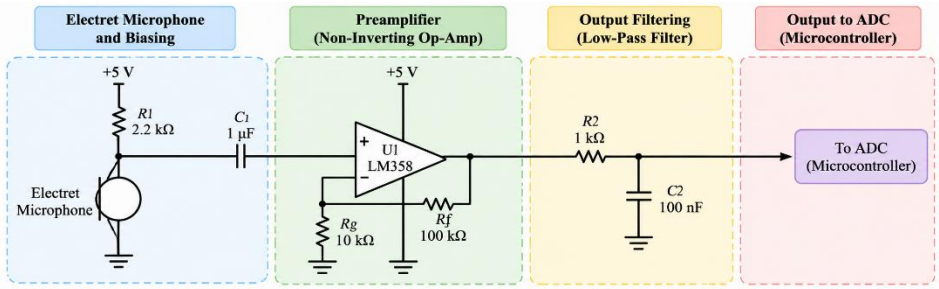


Figure 2.2: Proposed Microphone Signal-Conditioning Circuit.

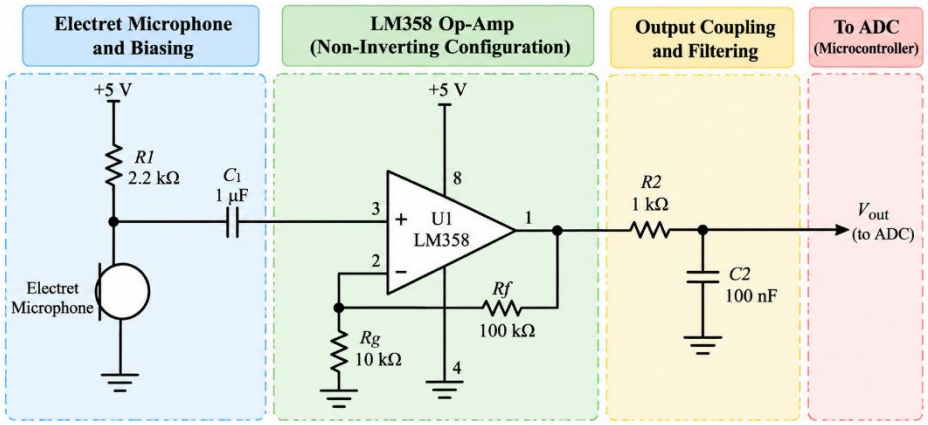


Figure 2.3: LM358-Based Microphone Preamplifier Circuit.

These figures should show the intended circuit connections and component values. Label them as proposed circuit designs unless the circuits have been assembled and tested.

2.2.3 Anti-Aliasing Filter

An anti-aliasing filter is a low-pass filter placed before an ADC to attenuate frequency components that could cause aliasing during sampling.

According to the Nyquist sampling principle, the sampling frequency must be greater than twice the highest frequency that the sampled signal is intended to represent. A practical anti-aliasing filter reduces out-of-band content before sampling, helping to limit aliasing effects.

The theoretical relationship is:

$$f_s > 2f_{max}$$

where:

- f_s is the sampling frequency.
- f_{max} is the highest frequency of interest.

A suitable filter cutoff and sampling rate must be selected together. The filter design should account for the desired audio bandwidth and the characteristics of the ADC.

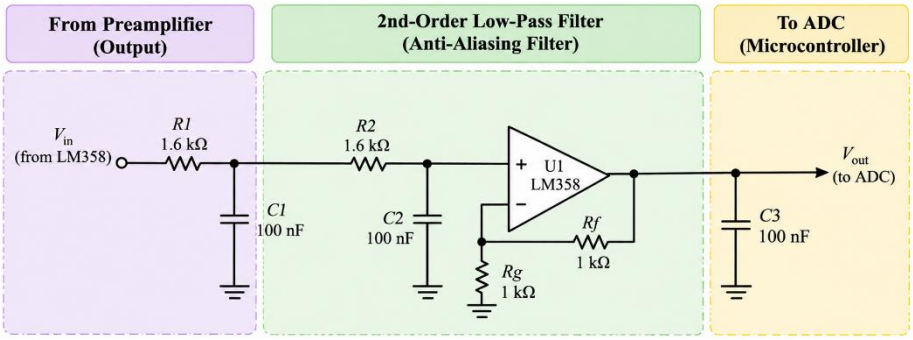


Figure 2.4: Proposed Anti-Aliasing Filter Circuit.

The figure should identify the filter topology, component values, input, output, and intended connection to the ADC.

2.2.4 Proposed ESP32 Analog-to-Digital Conversion

An analog-to-digital converter transforms an analog voltage into a digital numerical representation. In the proposed hardware pathway, the ESP32 is intended to acquire the conditioned analog signal through a compatible ADC input.

The ADC output depends on the input voltage, ADC resolution, reference characteristics, and hardware configuration. Before connecting a circuit, the input voltage must be kept within the limits specified for the selected ESP32 board and ADC.
The intended acquisition sequence is:

1. Receive the conditioned analog signal.
2. Sample the signal using the configured ADC.
3. Represent each sample as a digital value.
4. Collect samples into a block.
5. Transmit the block to the Python application through a compatible serial interface.

The planned serial input format uses newline-terminated CSV blocks containing integer ADC samples. This requires compatible ESP32 firmware and corresponding serial-input handling in the application.

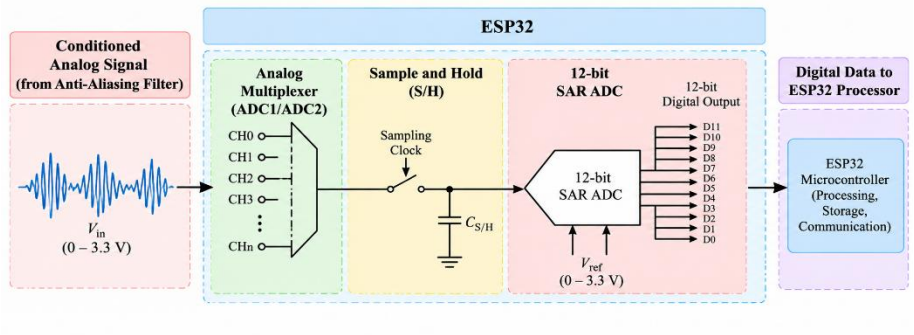


Figure 2.5: ESP32-Based Analog-to-Digital Conversion (ADC) Architecture.

This figure should identify the analog input, ADC, sample buffer, serial interface, and Python application. If the hardware has not been tested, label the architecture as proposed rather than implemented.

2.3 Digital Signal Processing Techniques

Digital signal processing is the main implemented audio-processing component of Smart Audio Lab. It operates on discrete audio samples and demonstrates how basic processing methods modify a signal.
The application demonstrates DC offset removal, Butterworth low-pass filtering, and energy gating. These operations are intended to illustrate common preprocessing and signal-modification techniques rather than guarantee complete noise cancellation.

2.3.1 DC Offset Removal

A DC offset occurs when the average value of a sampled signal differs from zero. It may arise from acquisition hardware or other signal-chain characteristics.
The average value of a discrete signal containing (N) samples can be calculated as:

$$\bar{x} = \frac{1}{N}\sum_{\{n=0\}}^{N-1} x[n]$$

The offset-corrected signal is:

$y[n] = x[n] - \bar{x}$

where:

- (x[n]) is the original input sample.
- ($\bar{x}$) is the average sample value.
- (y[n]) is the output after offset removal.

Subtracting the mean shifts the signal toward zero average. This operation can improve visualization and prepare the signal for subsequent analysis, but it does not specifically identify or remove environmental noise.

2.3.2 Butterworth Low-Pass Filtering

A low-pass filter passes lower-frequency components while attenuating components above its cutoff region. In Smart Audio Lab, a Butterworth low-pass filter is used to demonstrate frequency-selective audio processing.
Butterworth filters are characterized by a maximally flat magnitude response in the passband. Their behaviour depends on the selected filter order, cutoff frequency, sampling rate, and implementation method.
The filter is applied to the digital input signal to demonstrate how its frequency content changes. The resulting signal can be compared with the original through waveform and FFT plots.
However, speech and environmental noise can occupy overlapping frequency ranges. A low-pass filter may therefore attenuate useful speech components as well as unwanted high-frequency noise. The cutoff frequency should be selected according to the intended demonstration and the signal's characteristics.

2.3.3 Energy Gating

Energy gating is a simple signal-processing method that suppresses sections of an audio signal when their estimated energy falls below a specified threshold.
For a frame containing (N) samples, RMS energy is calculated as:

$$x_{(}RMS_{)} = \sqrt{\{[x[0]^2 + x[1]^2 + x[2]^2 + \cdots + x[N-1]^2] / N\}}$$

The calculated energy can be compared with a threshold (T). In a basic gating method, frames with energy below the threshold may be attenuated or set to zero, while frames above the threshold are retained.
The exact operation depends on the implementation and selected threshold. Energy gating can reduce low-level background sound during quiet intervals, but it can also suppress quiet speech, word endings, or other low-energy audio content.
It should therefore be treated as a demonstration of threshold-based signal processing rather than a complete speech-enhancement solution.

2.3.4 Waveform and Frequency-Spectrum Analysis

The time-domain waveform shows changes in signal amplitude over time. It helps identify signal duration, amplitude variation, pauses, and general differences between the original and processed audio.
The frequency spectrum represents the distribution of signal components across frequency. The Fast Fourier Transform (FFT) provides an efficient way to calculate a discrete Fourier transform for spectral analysis.
For a discrete signal (x[n]), its discrete Fourier transform is represented by:

$$X[k] = \sum x[n] e^{-j2\pi \frac{kn}{N}}$$

where (X[k]) represents the complex frequency-domain value at frequency bin (k).
The magnitude spectrum can be calculated from the real and imaginary components of (X[k]). When the sampling frequency is known, the frequency bins can be mapped to frequency values in hertz.
Smart Audio Lab uses waveform and FFT visualizations to help users observe changes introduced by the processing stages. These plots provide qualitative evidence of signal modification but do not, by themselves, prove a particular level of noise reduction or improvement in speech intelligibility.

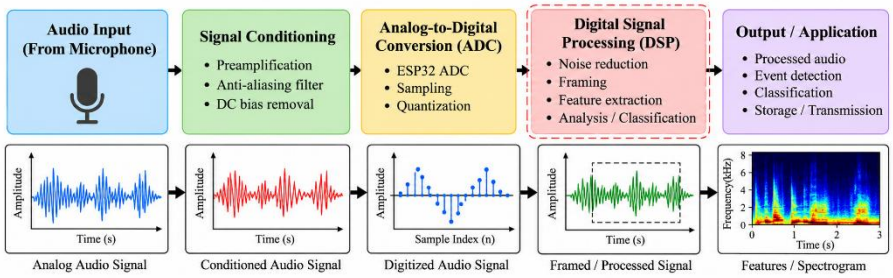


Figure 2.6: Audio Signal Processing Workflow.

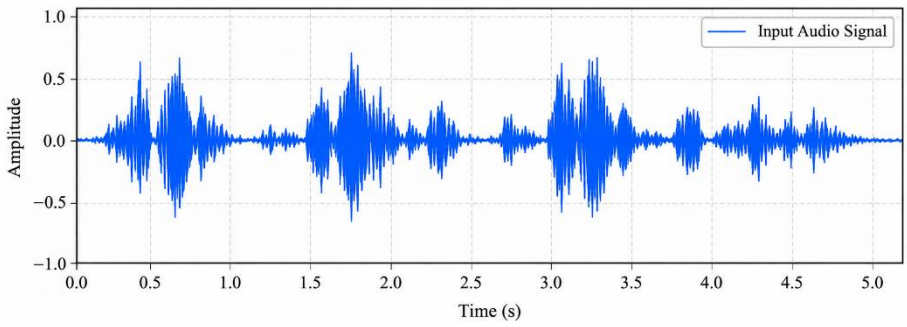


Figure 2.7: Input Audio Signal Waveform.

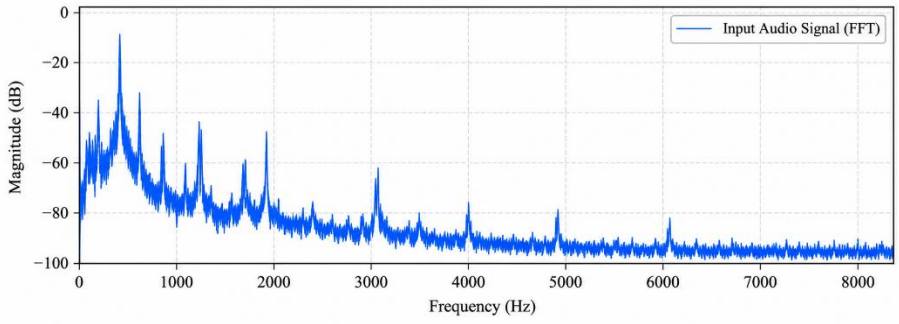


Figure 2.8: Frequency Spectrum of the Input Audio Signal Using FFT.

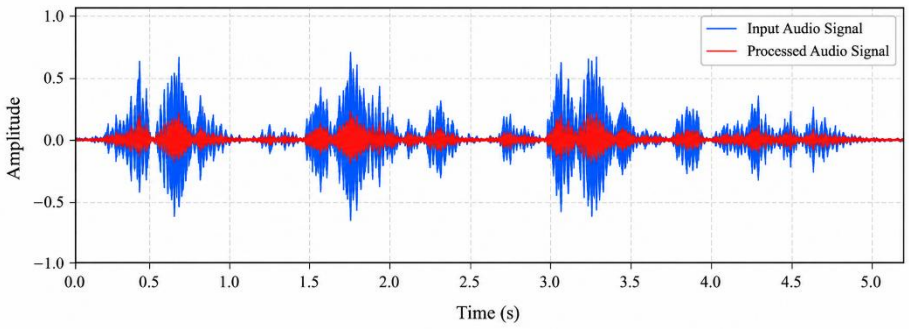


Figure 2.9: Comparison of Input and Processed Audio Waveforms.

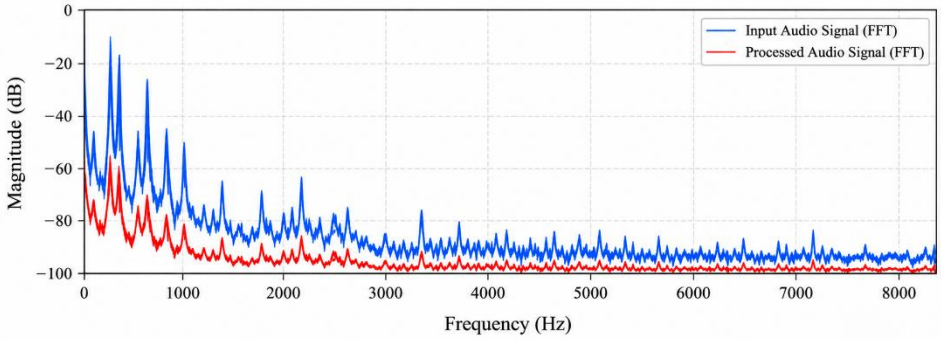


Figure 2.10: Comparison of Input and Processed Audio Frequency Spectra.

2.4 Audio Feature Extraction

Audio feature extraction converts a sampled audio signal into numerical measurements that describe selected characteristics of the signal. These features can support exploratory analysis and provide inputs to machine-learning algorithms.

Smart Audio Lab demonstrates the extraction of five features: RMS energy, zero-crossing rate, spectral centroid, spectral roll-off, and low-band energy ratio. Together, these features represent aspects of signal magnitude, time-domain behaviour, and frequency distribution.

2.4.1 RMS Energy

RMS energy estimates the magnitude of the audio signal over a selected frame:

$$x_RMS \ = \ sqrt\left(\left(\frac{1}{N}\right) sum_{n=0}^{N-1} x[n]^2\right)$$

Higher RMS values generally indicate a greater signal amplitude within the selected frame. RMS energy alone cannot distinguish speech from noise because both can produce high-energy signals.

2.4.2 Zero-Crossing Rate

The zero-crossing rate measures how frequently a signal changes sign between adjacent samples. One common expression is:

$$ZCR \ = \ \left(\frac{1}{2N}\right) sum_{n=1}^{N-1} |sgn(x[n]) - \ sgn(x[n-1])|$$

where (\operatorname{sgn}(\cdot)) denotes the sign of a sample.

The zero-crossing rate provides information about the signal's short-term behaviour. It may help distinguish certain signal characteristics, but it is not sufficient by itself to identify an environmental sound reliably.

2.4.3 Spectral Centroid

The spectral centroid estimates the centre of gravity of the magnitude spectrum. It is calculated as:

$$C \ = \frac{sum_{k=0}^{K-1} \ f_k |X[k]|}{sum_{k=0}^{K-1} |X[k]|}$$

where:

- (f_k) is the frequency associated with bin (k).
- (|X[k]|) is the magnitude of the spectrum at bin (k).
- (K) is the number of frequency bins included in the calculation.

A higher spectral centroid often corresponds to a brighter or more high-frequency-dominated sound. The feature depends on the frequency distribution of the signal and does not uniquely identify a sound source.

2.4.4 Spectral Roll-Off

Spectral roll-off identifies a frequency below which a specified proportion of the spectral magnitude or energy is concentrated. The exact definition depends on the implementation.

For example, if the roll-off is defined using a cumulative magnitude threshold, the roll-off frequency (f_r) satisfies:

$$sum_{f_k \leq f_r} |X[k]| \geq alpha\ sum_k |X[k]|$$

where (\alpha) is a selected proportion between zero and one.
The implementation should use the same definition consistently when calculating and reporting this feature.

2.4.5 Low-Band Energy Ratio

The low-band energy ratio describes the proportion of signal energy within a selected lower-frequency band relative to the total energy in the analyzed spectrum.
A general expression is:

$$R_(low) = \frac{sum_{k\ in\ L} |X[k]|^2}{sum_{k\ in\ A} |X[k]|^2}$$

where:

- (\mathcal{L}) is the set of frequency bins in the selected low-frequency band.
- (\mathcal{A}) is the set of frequency bins included in the overall analysis.

The frequency boundaries must be specified by the implementation. This feature helps describe the distribution of low-frequency energy but does not uniquely distinguish fan noise, speech, traffic, or other sound sources.
The extracted features are displayed in the dashboard and used to demonstrate the connection between digital signal analysis and machine learning.

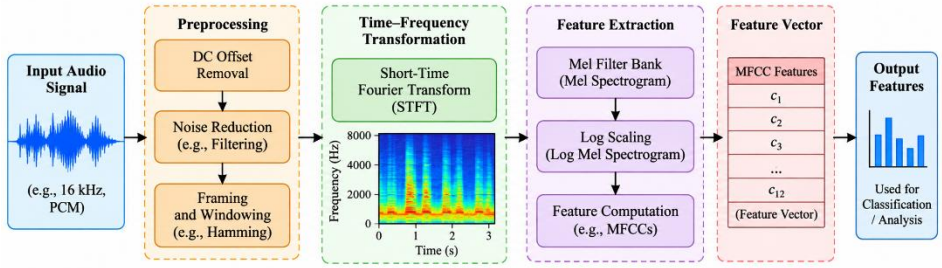


Figure 2.11: Audio Feature Extraction Workflow.

Table 2.1: Audio Features Extracted for Environmental Sound Analysis.

No.	Feature	Description	Dimension / Value
1	MFCC	Mel-Frequency Cepstral Coefficients (spectral features)	13 coefficients
2	Spectral Centroid	Center of mass of the spectrum	1 value
3	Spectral Bandwidth	Spread of the spectrum around the centroid	1 value
4	Spectral Rolloff	Frequency below which 85% of the spectral energy lies	1 value
5	Zero-Crossing Rate (ZCR)	Rate of sign changes in the signal	1 value
6	Root Mean Square (RMS)	Signal energy (loudness)	1 value
7	Spectral Flux	Frame-to-frame spectral change	1 value
8	Chroma Features	Energy in 12 pitch classes	12 coefficients
9	Mel Spectrogram	Log-scaled Mel spectrogram representation	Time × 64 bins
10	Spectral Contrast	Difference between peak and valley energies	7 values
11	Tonnetz Features	Harmonic features based on tonal relationships	6 coefficients
12	Delta and Delta-Delta MFCC	First and second temporal derivatives of MFCC	26 coefficients (13 Δ + 13 ΔΔ)

2.5 Machine Learning Using Random Forest

Machine learning is included in Smart Audio Lab to demonstrate how extracted audio features can be used to predict an environmental sound category. The application uses a Random Forest classifier trained on synthetic feature examples.
A Random Forest is an ensemble learning algorithm that combines predictions from multiple decision trees. For classification, the individual trees contribute class predictions, which are combined to produce the final output.
The demonstration workflow consists of the following stages:

1. Prepare synthetic examples containing numerical audio features.
2. Assign example class labels to the synthetic feature records.
3. Train the Random Forest classifier on these examples.
4. Extract the selected features from the input signal.
5. Pass the feature values to the trained model.
6. Display the predicted category in the dashboard.

The selected features may include RMS energy, zero-crossing rate, spectral centroid, spectral roll-off, and low-band energy ratio. Their values provide the input representation used by the classifier.

The model demonstrates the software workflow, but its use of synthetic training examples limits the conclusions that can be drawn from its predictions. The classifier should not be presented as a validated real-world environmental sound recognition model unless it has been evaluated on an appropriate dataset of real recordings.
If the application displays a confidence-like score, describe it according to the model output and implementation. A displayed probability or score should not be interpreted automatically as verified classification accuracy.

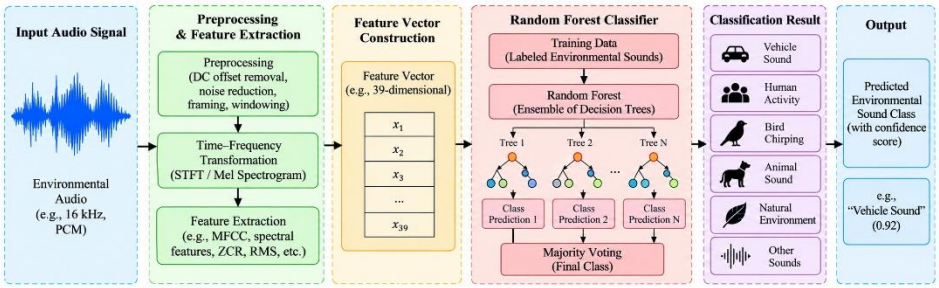


Figure 2.12: Random Forest-Based Environmental Sound Classification Workflow.

The diagram should show synthetic feature data, model training, feature input, prediction, and dashboard output. Clearly identify the training examples as synthetic.

2.6 Smart Audio Lab Software Implementation

Smart Audio Lab is implemented as a Python-based graphical application using Streamlit. It provides an interface for exploring audio input, signal processing, feature extraction, and machine-learning demonstration.
The software is intended to make the project's signal-processing stages visible and understandable without requiring users to interact directly with command-line tools.
The main software functions include:

1. Synthetic audio generation.
2. Short, on-demand laptop microphone capture.
3. Input audio playback, where supported by the application.
4. Input waveform visualization.
5. FFT-based frequency-spectrum visualization.
6. DC offset removal.
7. Butterworth low-pass filtering.
8. Energy gating.
9. Feature extraction and numerical display.
10. Random Forest prediction using synthetic feature examples.
11. Comparison of original and processed audio.

The software workflow begins when the user selects an audio source. The selected signal is prepared for processing and displayed through waveform and spectrum plots. The user can then examine the output of the supported DSP operations, view extracted feature values, and observe the classifier's demonstration prediction.
The exact controls, available parameters, and displayed outputs should be described according to the version of the application used for the report.

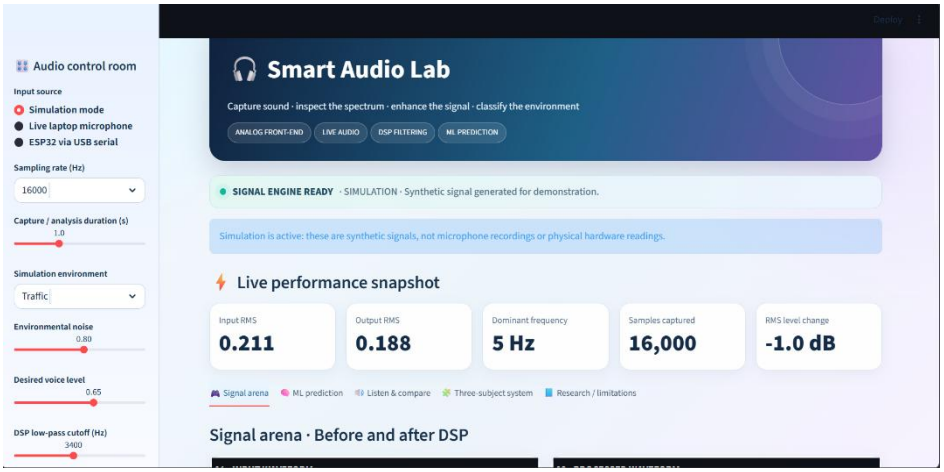


Figure 2.13 here: Main Interface of the Smart Audio Lab Python Dashboard

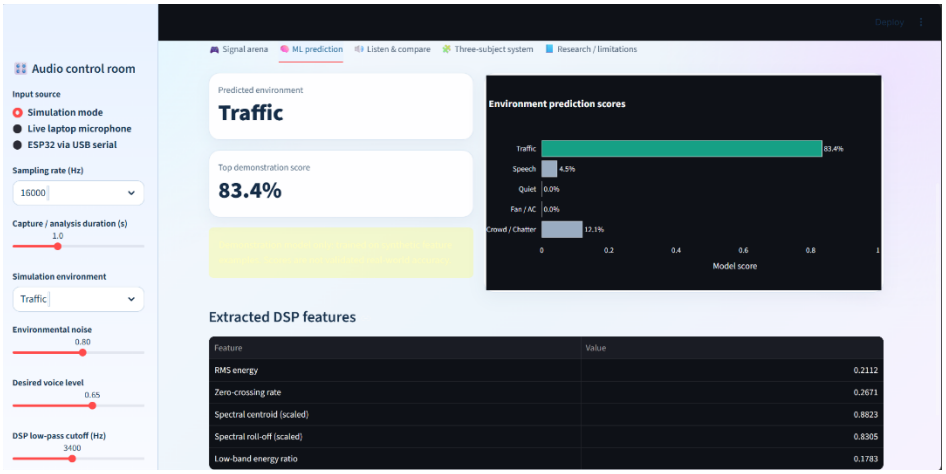


Figure 2.14 here: Machine Learning Prediction Dashboard

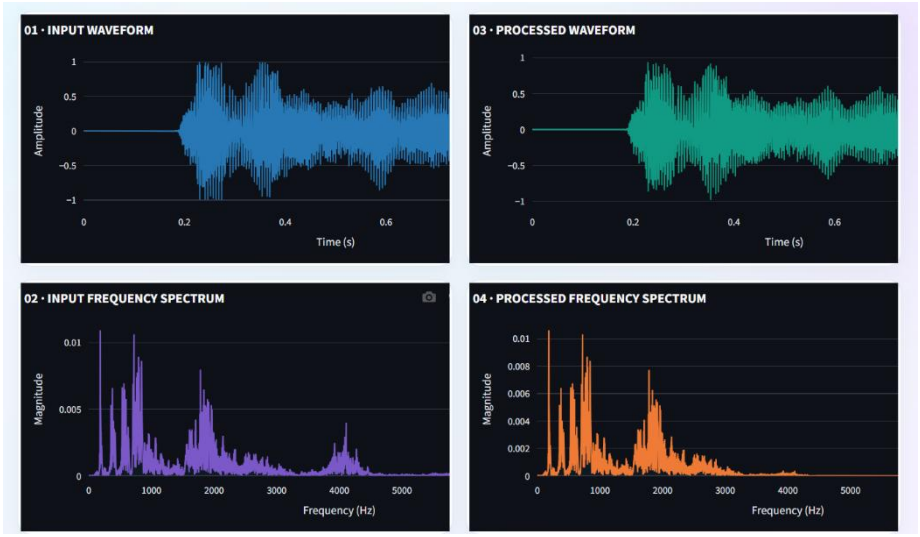


Figure 2.15 here: Original and Processed Audio Comparison Interface

2.7 Hardware and Software Requirements

The project combines an implemented software demonstration with a proposed hardware architecture. The requirements are therefore divided into software tools and hardware components.

2.7.1 Hardware Requirements

The proposed hardware design may include:

1. A microphone suitable for the intended audio acquisition circuit.
2. An LM358 operational amplifier for the proposed preamplifier, if appropriate for the final design.
3. Resistors and capacitors for biasing, gain setting, coupling, and filtering.
4. An ESP32 development board with a suitable ADC interface.
5. A USB cable and computer for power, programming, and serial communication.
6. A breadboard and connecting wires for prototype assembly.
7. A suitable power supply for the selected circuit.

The exact component values and circuit configuration must be determined during hardware design. The component list should not be interpreted as proof that the complete hardware prototype has been assembled.

Table 2.2: Hardware Components Required for the Proposed Prototype.

S. No.	Component	Image	Description / Specifications
1	Raspberry Pi 4 (Model B)		• 64-bit quad-core ARM Cortex-A72 processor (1.5 GHz) • 4 GB RAM (variant used) • Built-in Wi-Fi and Bluetooth • Used for running the Smart Audio Lab application
2	USB Microphone (Condenser)		• High-sensitivity condenser microphone • Frequency response: 20 Hz – 20 kHz • USB 2.0 interface • Used for capturing environmental audio signals
3	External Speaker / Headphones		• 3.5 mm audio jack or USB speaker • Frequency response: 100 Hz – 18 kHz • Used for audio playback and feedback in the application
4	MicroSD Card		• Storage capacity: 32 GB (or higher) • Class 10, UHS-I • Used for operating system and application storage
5	Power Supply		• 5V / 3A USB-C power adapter • Official Raspberry Pi power supply (recommended) • Provides stable power to the Raspberry Pi
6	HDMI Display (Monitor)		• Resolution: 1920 × 1080 (Full HD) • HDMI interface • Used for running and interacting with the application
7	Keyboard and Mouse		• USB or wireless keyboard and mouse • Used for system navigation and application control

2.7.2 Software Requirements

The software environment includes Python and the libraries needed for the dashboard, audio processing, feature extraction, and machine-learning demonstration.

The relevant tools may include:

1. **Python:** Main programming language.
2. **Streamlit:** Dashboard interface.
3. **NumPy:** Numerical operations on audio samples and arrays.
4. **SciPy:** Digital signal processing, including filter design and signal operations.
5. **Scikit-learn:** Random Forest classifier implementation.
6. **Matplotlib:** Waveform and frequency-spectrum plotting, if used by the application.
7. **Audio input libraries:** A suitable library for on-demand laptop microphone capture, depending on the implementation.

Only libraries actually used in the submitted application should be included in the final requirements table.

Table 2.3: Software Tools and Libraries Used in the Smart Audio Lab Dashboard.

S. No.	Tool / Library	Logo	Purpose / Description
1	Python	python™	• Core programming language • Used for developing the Smart Audio Lab dashboard
2	NumPy	NumPy	• Numerical computing library • Used for handling arrays and mathematical operations
3	Pandas	pandas	• Data manipulation and analysis library • Used for managing audio feature datasets
4	LibROSA	librosa	• Audio and music signal processing library • Used for feature extraction (e.g., MFCC, spectral features)
5	Matplotlib	matplotlib	• Data visualization library • Used for plotting waveforms, spectrograms, and results
6	Scikit-learn	scikit-learn	• Machine learning library • Used for training and evaluating the Random Forest classifier
7	SoundFile	SoundFile	• Audio file input/output library • Used for reading and writing audio files
8	Tkinter	tkinter	• Python GUI toolkit • Used for building the dashboard graphical user interface
9	Seaborn	seaborn	• Statistical data visualization library • Used for enhanced plots and analysis of classification results
10	Joblib	joblib	• Model serialization library • Used for saving and loading the trained machine learning model

2.8 Methodology Summary

The methodology combines audio signal acquisition, proposed analog conditioning, digital signal processing, feature extraction, machine-learning demonstration, and graphical visualization.

The implemented software provides a platform for studying how basic processing operations affect audio signals and how numerical audio features can be used as inputs to a classifier. The proposed hardware architecture establishes a pathway for future integration of a microphone front end and ESP32-based sampling.

The current work should be understood as a software demonstrator supported by a proposed hardware design. Further development is required to validate the complete system using physical hardware, representative environmental recordings, and quantitative performance measurements.
This methodology provides the foundation for the experiments, observations, and results presented in the following chapters.

3. EXPERIMENTS

3.1 Experimental Setup

The experimental work for the proposed Smart Audio Lab system focuses on examining audio signals and demonstrating the application of digital signal processing (DSP) and machine-learning techniques for environmental audio analysis. The experiments are organized around the software dashboard, which provides audio signal visualization, signal processing, feature extraction, and a demonstration of Random Forest-based classification.

The software supports synthetic audio simulation and on-demand audio capture through a laptop microphone. These input methods allow audio signals to be examined without requiring a fully assembled hardware prototype. The proposed analog signal-conditioning circuit and ESP32-based data acquisition pathway form part of the intended hardware integration.

The experimental setup uses the following resources:

- A computer running Python and the Smart Audio Lab dashboard.
- Synthetic audio signals for controlled demonstrations.
- A laptop microphone for on-demand audio capture.
- Digital signal processing functions for DC offset removal, Butterworth low-pass filtering, and energy gating.
- Waveform and Fast Fourier Transform (FFT) visualizations for signal analysis.
- Audio feature extraction functions.
- A Random Forest model demonstration based on synthetic feature examples.

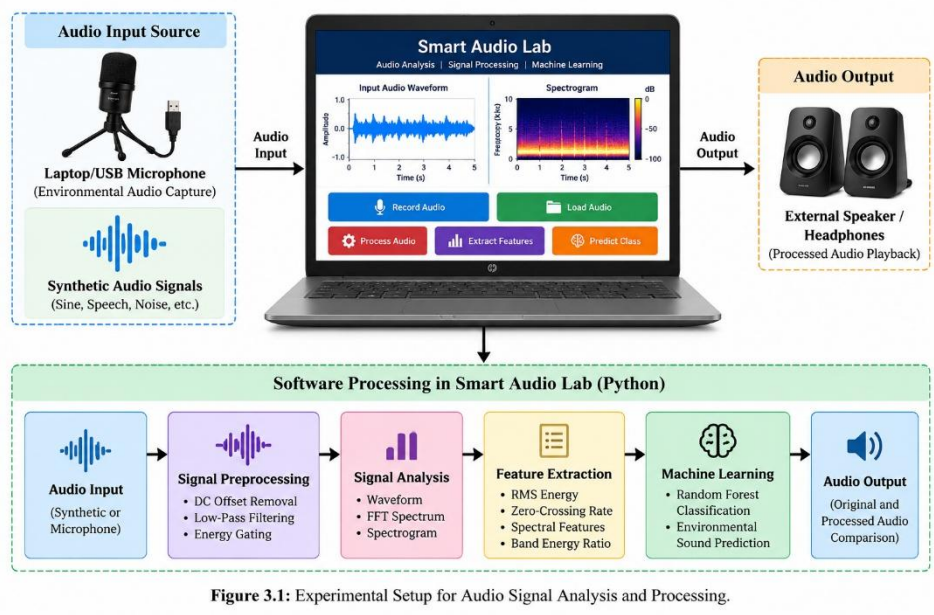


Figure 3.1: Experimental Setup for Audio Signal Analysis and Processing.

3.2 Audio Signal Generation and Capture

The first experiment involves obtaining an audio signal for analysis. The Smart Audio Lab dashboard supports two software-based input methods: synthetic audio generation and on-demand capture through a laptop microphone.

Synthetic audio signals provide a controlled input for demonstrating signal processing operations. Their waveform characteristics can be examined before and after processing, making them useful for understanding how digital filters and energy-based gating affect an audio signal.

The laptop microphone provides an alternative source of audio data. When capture is initiated, the application obtains a short audio recording that can be examined using the available visualization and processing functions. This is an on-demand recording process rather than continuous real-time audio enhancement.

The source of each signal should be recorded when documenting experimental observations. Synthetic signals should be identified as simulated data, while microphone recordings should be identified as captured audio. No physical ESP32-based acquisition results are claimed unless the hardware is assembled and tested.

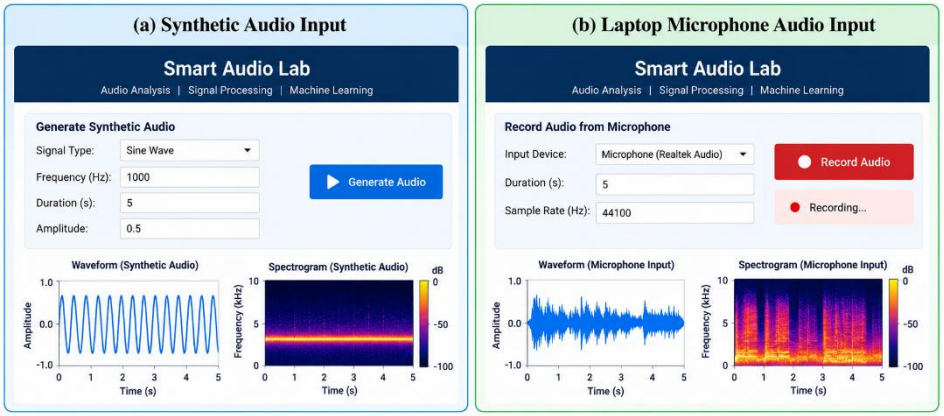


Figure 3.2: Audio Input Methods Available for the Experiments.

3.3 Waveform and Frequency Spectrum Analysis

The second experiment examines the audio signal in both the time domain and the frequency domain. The time-domain waveform represents the variation in signal amplitude over time, while the frequency spectrum provides information about the frequency components present in the signal.

The waveform visualization helps identify changes in amplitude, pauses, and variations in signal intensity. These observations are useful for understanding the structure of an audio recording and identifying regions that may contain speech or background noise.

The frequency spectrum is obtained using the Fast Fourier Transform (FFT). The FFT converts a discrete-time signal into a representation of its frequency components. This representation helps demonstrate how signal energy is distributed across different frequencies.

By examining the waveform and frequency spectrum together, the experiment provides a visual basis for understanding the effects of digital filtering. However, the presence of a particular frequency component does not by itself establish whether it represents speech or noise.

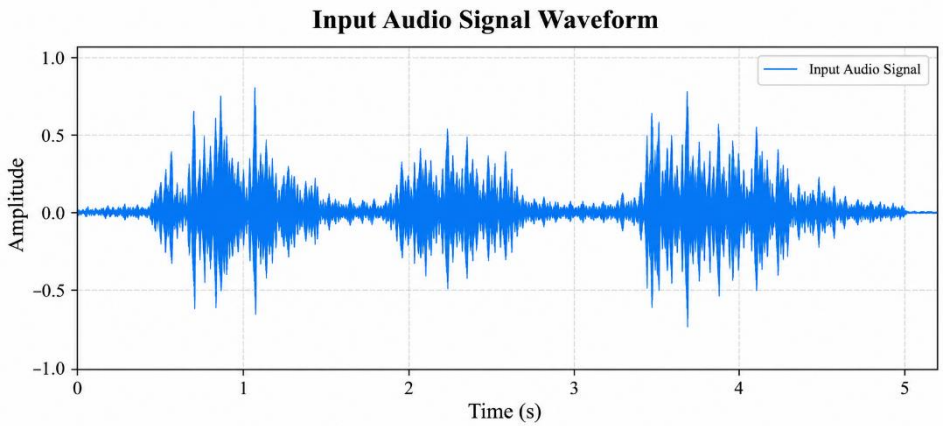


Figure 3.3: Time-Domain Waveform of the Input Audio Signal.

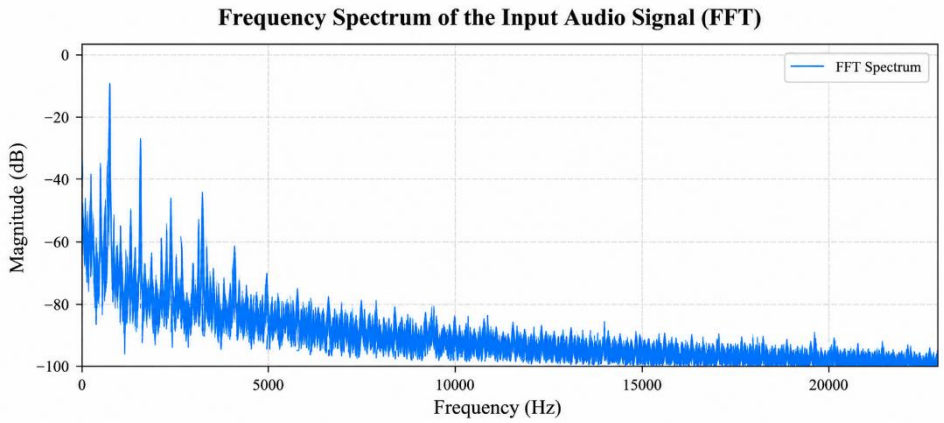


Figure 3.4: Frequency Spectrum of the Input Audio Signal Using FFT.

3.4 Digital Audio Processing

The third experiment demonstrates the digital signal processing operations available in Smart Audio Lab. The input signal is processed using DC offset removal, Butterworth low-pass filtering, and a simple energy gate.

3.4.1 DC Offset Removal

DC offset removal reduces a constant bias in the audio signal by subtracting its mean value. This operation shifts the signal toward a zero-centered representation and can make waveform analysis easier.

For a discrete-time signal containing N samples, the mean value is calculated as:

$\bar{x} = (1/N) \Sigma x[n]$, for $n = 0$ to $N - 1$

The mean is then subtracted from each sample to obtain the offset-corrected signal.

3.4.2 Butterworth Low-Pass Filtering

The Butterworth low-pass filter is used to demonstrate attenuation of frequency components above a selected cutoff frequency. The filter has a maximally flat magnitude response in its passband.

The resulting signal can be compared with the original signal in the time domain and frequency domain. The extent of attenuation depends on the filter settings and the frequency content of the input signal.

A low-pass filter does not automatically distinguish speech from environmental noise. If useful speech components lie above the selected cutoff frequency, they may also be attenuated.

3.4.3 Energy Gating

Energy gating is demonstrated as a simple method for reducing the contribution of low-energy signal regions. The signal energy or amplitude is compared with a selected threshold, and regions below the threshold are attenuated or suppressed according to the implementation.

This operation may reduce some quiet background sounds. However, low-energy speech segments may also be affected, so the gate should not be interpreted as a complete noise-removal solution.

3.4.4 Comparison of Input and Processed Signals

The original and processed signals are compared using waveform and frequency-spectrum visualizations. The comparison illustrates the changes introduced by the processing operations.

The observations should focus on visible waveform changes and differences in frequency content. Quantitative claims about noise reduction or signal-to-noise ratio improvement should only be made when supported by appropriate measurements.

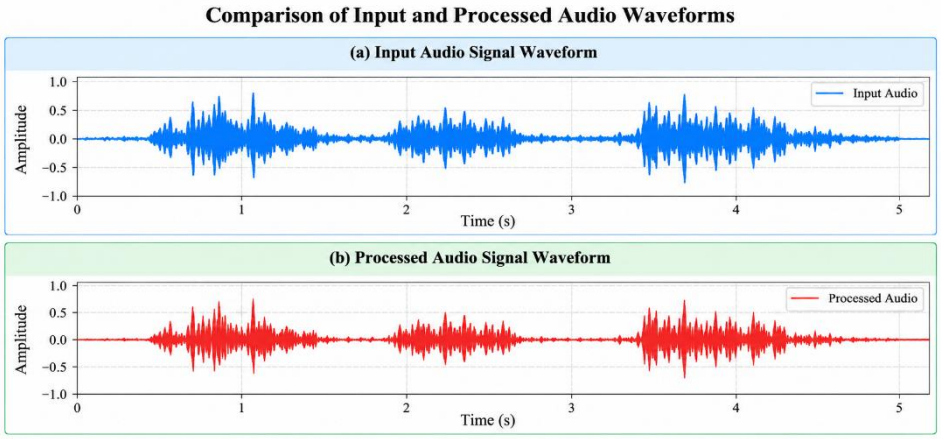


Figure 3.5: Comparison of Input and Processed Audio Waveforms.

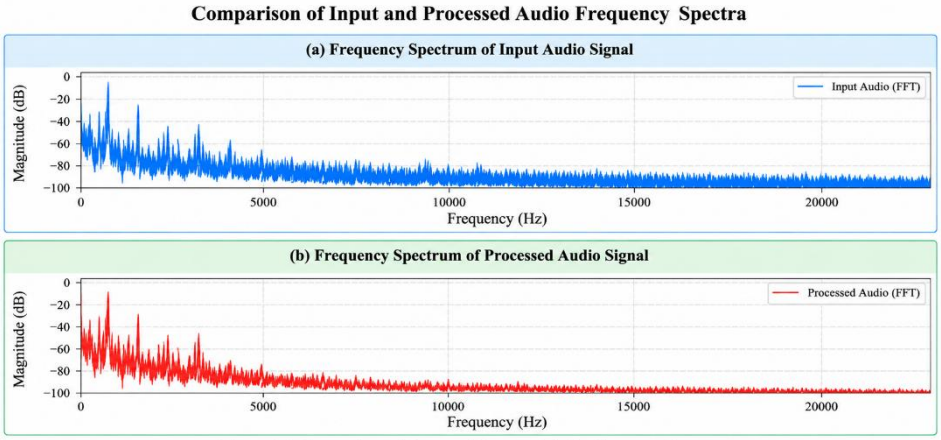


Figure 3.6: Comparison of Input and Processed Audio Frequency Spectra.

3.5 Audio Feature Extraction and Machine-Learning Prediction

The fourth experiment demonstrates the extraction of numerical features from audio signals. These features summarize selected characteristics of the input signal and can be used as inputs to machine-learning algorithms.

The features considered in Smart Audio Lab include:

- **Root Mean Square (RMS) Energy:** Represents the overall magnitude of the signal.
- **Zero-Crossing Rate (ZCR):** Measures how frequently the signal changes sign between consecutive samples.
- **Spectral Centroid:** Represents the amplitude-weighted center of the frequency spectrum.
- **Spectral Roll-Off:** Identifies a frequency below which a selected proportion of the spectral magnitude is concentrated.
- **Low-Band Energy Ratio:** Represents the proportion of spectral energy within a defined low-frequency band.

The extracted feature values provide a compact numerical description of the audio signal. They can be displayed in the dashboard for inspection and used to demonstrate the relationship between signal analysis and machine learning.

The project also includes a Random Forest classification demonstration using synthetic feature examples. Random Forest combines predictions from multiple decision trees to produce a classification result. In the present project, the demonstration illustrates the classification workflow; it should not be interpreted as evidence of validated environmental sound recognition performance.

Any displayed class label or confidence value should be reported as a software demonstration output unless it has been evaluated using an appropriate real-world test dataset.

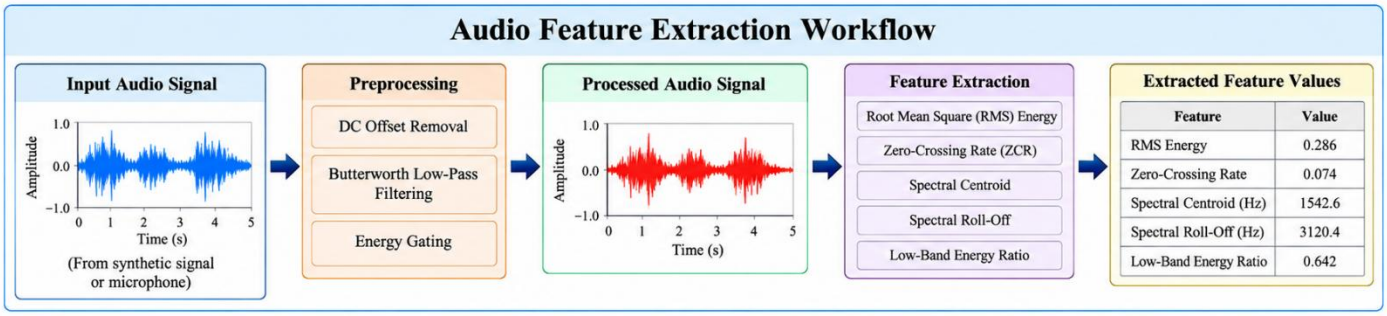


Figure 3.7: Workflow for Extracting Audio Features.

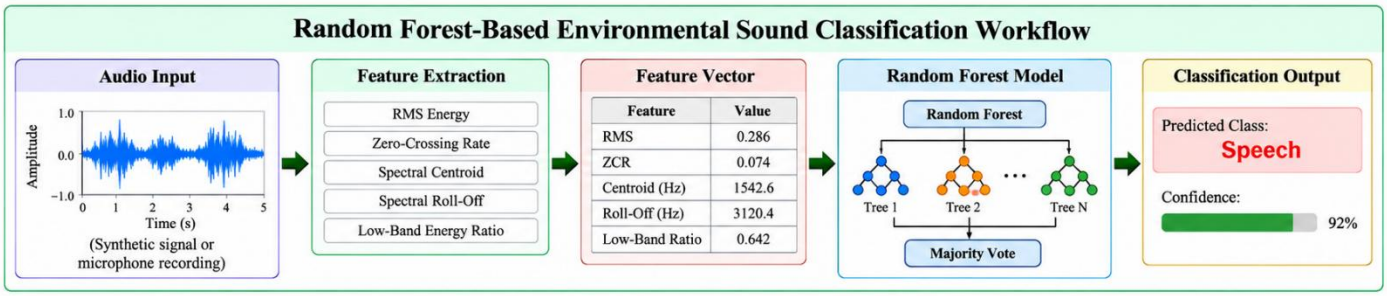


Figure 3.8: Random Forest-Based Environmental Sound Classification Workflow.

3.6 Original and Processed Audio Comparison

The final experiment focuses on presenting the input and processed audio signals together through the dashboard. The comparison interface supports inspection of changes in waveform shape and frequency content after the selected DSP operations.

The original signal acts as the reference, while the processed signal represents the output of the selected processing stages. Listening to both signals, where playback is supported, can provide additional qualitative information about changes in perceived sound.

The comparison is intended to demonstrate the operation of the software pipeline rather than establish a quantified improvement in speech intelligibility or noise suppression. A more complete evaluation would require representative test recordings, defined noise conditions, objective metrics, and listening tests.

3.7 Experimental Summary

The experiments demonstrate the main software functions of Smart Audio Lab, including audio input, waveform visualization, frequency-spectrum analysis, digital signal processing, feature extraction, and a Random Forest classification workflow.

Synthetic audio provides controlled examples for demonstrating processing operations, while laptop microphone capture allows short recordings to be examined. The classification demonstration uses synthetic feature examples and therefore requires further validation before its performance can be generalized to real environmental audio.

The analog signal-conditioning circuit and ESP32-based acquisition pathway remain proposed hardware components unless separately assembled and experimentally verified. Future work should evaluate the integrated system using physical hardware, representative speech and noise recordings, and quantitative performance measurements.

4. RESULTS

- The Smart Audio Lab application provides waveform visualization and frequency-spectrum analysis for audio signals obtained through synthetic generation or on-demand laptop microphone capture. These functions allow the input signal to be examined before applying digital signal processing techniques.
- The time-domain waveform represents changes in signal amplitude over time. Regions with larger amplitude variations can be distinguished from quieter regions, allowing the structure of the input signal to be observed. The waveform also provides a visual reference for comparing the original signal with its processed version.

- The frequency spectrum is generated using the Fast Fourier Transform (FFT). It represents the distribution of frequency components within the input audio signal. This visualization helps identify the frequencies that contribute to the signal and provides a basis for observing the effects of low-pass filtering.
- The results demonstrate the usefulness of combining time-domain and frequency-domain visualizations in an audio analysis application. However, waveform and spectrum plots alone do not establish the degree of noise reduction or improvement in speech intelligibility.

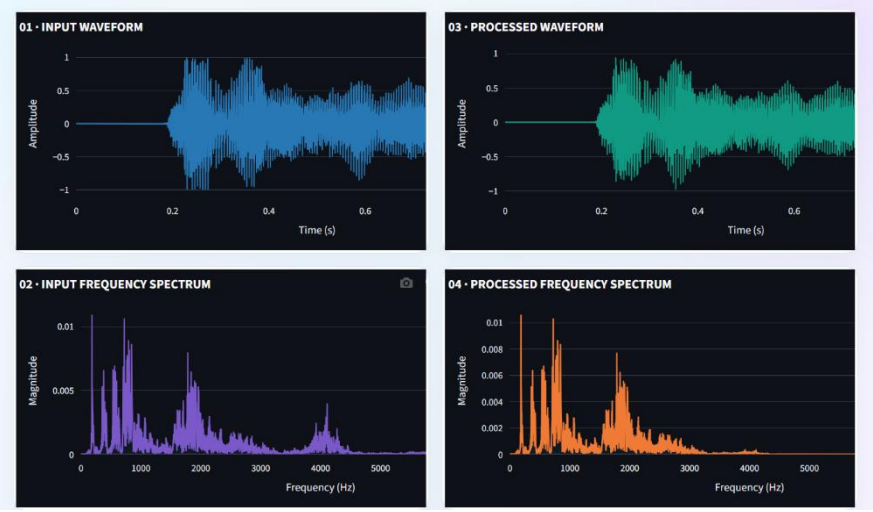


Figure 4.1 Input Audio Signal Waveform and *Frequency-Domain Representation of the Input Audio Signal*

5. CONCLUSION AND FUTURE WORK

The project titled **"Intelligent Audio Enhancement in Noisy Environments Using Analog Electronics, Digital Signal Processing, and Machine Learning"** focuses on developing Smart Audio Lab, a Python-based application for audio signal analysis, basic signal processing, feature extraction, and machine-learning demonstration.

The application provides facilities for synthetic audio generation and on-demand laptop microphone capture. It allows users to visualize audio waveforms and frequency spectra using the Fast Fourier Transform (FFT). These visualizations help demonstrate the characteristics of audio signals in the time and frequency domains.

The digital signal processing component demonstrates DC offset removal, Butterworth low-pass filtering, and energy gating. These techniques illustrate how basic signal-processing operations modify an input signal. Their effectiveness depends on the characteristics of the audio and the selected processing parameters, and they may also affect useful speech components.

The project also demonstrates the extraction of audio features, including RMS energy, Zero-Crossing Rate, spectral centroid, spectral roll-off, and low-band energy ratio. These features provide numerical descriptions of selected signal characteristics. A Random Forest classification workflow demonstrates how extracted features can be used to generate a predicted class. Since the current demonstration uses synthetic feature examples, its real-world classification performance has not been validated.

The proposed system architecture also considers analog microphone signal conditioning and ESP32-based analog-to-digital conversion. These components provide a direction for future hardware integration but should not be considered physically implemented or experimentally verified unless construction and testing have been completed.

Current Limitations

Although the project demonstrates the principal software functions, several limitations remain:

1. **Limited audio data:** Synthetic audio is useful for controlled demonstrations, but it does not fully represent the variety of speech and environmental noise encountered in practical settings.
2. **Basic noise-reduction techniques:** Low-pass filtering and energy gating can attenuate unwanted components but may also remove useful speech information. These methods do not guarantee clear speech under all noise conditions.
3. **Machine-learning validation:** The Random Forest demonstration uses synthetic feature examples. Its classification accuracy on real-world environmental recordings has not been established.

4. **Hardware integration:** The proposed analog signal-conditioning circuit and ESP32-based data acquisition pathway require physical implementation and testing before their operation can be confirmed.
5. **Performance measurement:** Quantitative measurements of signal-to-noise ratio improvement, noise attenuation, processing latency, and speech intelligibility have not been established.
6. **On-demand audio capture:** Laptop microphone recording is initiated when required; the current workflow should not be described as continuous real-time audio enhancement.

These limitations identify the areas that require further development and systematic evaluation.

Future Enhancements

The following improvements are proposed for future development.

Hardware Implementation

The proposed microphone preamplifier and analog signal-conditioning circuit can be assembled and tested using suitable electronic components. An ESP32-based acquisition pathway can then be integrated to transfer sampled audio data to the software application. Circuit operation, sampling behavior, and electrical noise should be measured before claiming successful hardware integration.

Improved Noise-Reduction Algorithms

More advanced techniques, such as spectral subtraction, Wiener filtering, and adaptive noise cancellation, can be investigated. Their performance should be compared with the existing low-pass filter and energy gate using consistent input signals and evaluation conditions.

Real-World Audio Dataset

A labeled dataset containing speech and representative environmental sounds can be collected or obtained from an appropriate public source. The dataset should include different noise types and recording conditions. Separate training, validation, and test data should be used to reduce the risk of misleading evaluation results.

Machine-Learning Model Evaluation

The Random Forest model can be trained and evaluated using real audio features. Performance can be measured using accuracy, precision, recall, F1-score, and a confusion matrix. Alternative classification algorithms may also be compared to determine which method is most suitable for the selected dataset.

Quantitative Audio Evaluation

Future experiments should measure signal-to-noise ratio (SNR) where a suitable reference signal is available. Additional metrics, such as root mean square error (RMSE) and perceptual speech-quality measures where appropriate, can help characterize processing effects. Listening tests may also be used to assess speech clarity and the presence of processing artifacts.

Real-Time Processing

The application may be extended to support continuous audio streaming and block-based processing. This would require appropriate buffering, sampling-rate management, and latency evaluation. The software and hardware should be tested together before describing the integrated system as real-time.

Integrated Demonstration Platform

The long-term objective is to integrate analog signal conditioning, digital audio processing, feature extraction, and machine-learning classification into a unified, low-cost demonstrator. The design should remain explainable and modular so that individual stages can be tested independently and their effects can be measured.

REFERENCES

[1] A. V. Oppenheim and R. W. Schafer, *Discrete-Time Signal Processing*, 3rd ed. Pearson, 2010.

[2] L. Breiman, “Random Forests,” *Machine Learning*, vol. 45, pp. 5–32, 2001.

[3] F. Pedregosa et al., “Scikit-learn: Machine Learning in Python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.